

Windows Hashes & Attacks : LM, NT

 medium.com/@AnisO./windows-authentication-attacks-lm-nt-aka-ntlm-794bdcfe3887

Anis Ouersighni

January 8, 2024

Since I'm now keen on learning about active directory, and I'm already building my own Local Labs, I'm always discovering a Lot of New Concepts within Windows, that can seem complex to understand, so I thought why not sharing my Notes of what I learned from different resources.

This article will be the first part of this "Windows Hashes & Attacks" serie, and we will discuss **the difference between LM and NT Hashes**.

Lan Manager (LM) Protocol & the Attacks :

LM Hash was the original way of **storing passwords** in a windows system. But that was weak and not enough. Soon enough it was deprecated. However it's stored in some machines for backwards compatibility.

It's **disabled by default since** windows vista/windows server 2008.

LM Hash was a weak hashing algorithm for many reasons, You will figure these reasons out once You know how LM hashing works.

Let's take a Look at **LM Hash Generation Steps** :

Let's assume that the user's password is **PassWord**

1 — All characters will be converted to upper case

PassWord -> PASSWORD

2 — In case the password's length is less than 14 characters it will be **padded with null characters**, so its length becomes 14, so the result will be

PASSWORD000000

3 — These 14 characters will be split into 2 halves

PASSWOR

D000000

4 — Each half is converted to bits, and after every 7 bits, a parity bit (0) will be added, so the result would be a 64 bits key for each half.

1101000011 -> 11010000011

As a result, **we will get two keys from the 2 pre-generated halves after adding these parity bits**

5 — **Each of these keys** is then used to encrypt the string "KGS!@#\$\$%" using DES algorithm in ECB mode so that the result would be

PASSWOR = E52CAC67419A9A22

D000000 = 4A3B108F3FA6CB6D

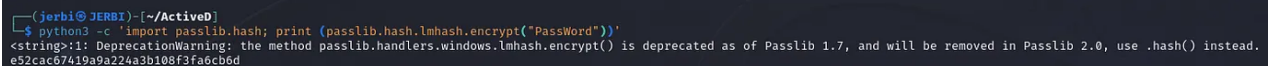
6 — The output of the two halves is then combined, and that makes out LM hash

E52CAC67419A9A224A3B108F3FA6CB6D

NB : you can do all the steps by using python with this line

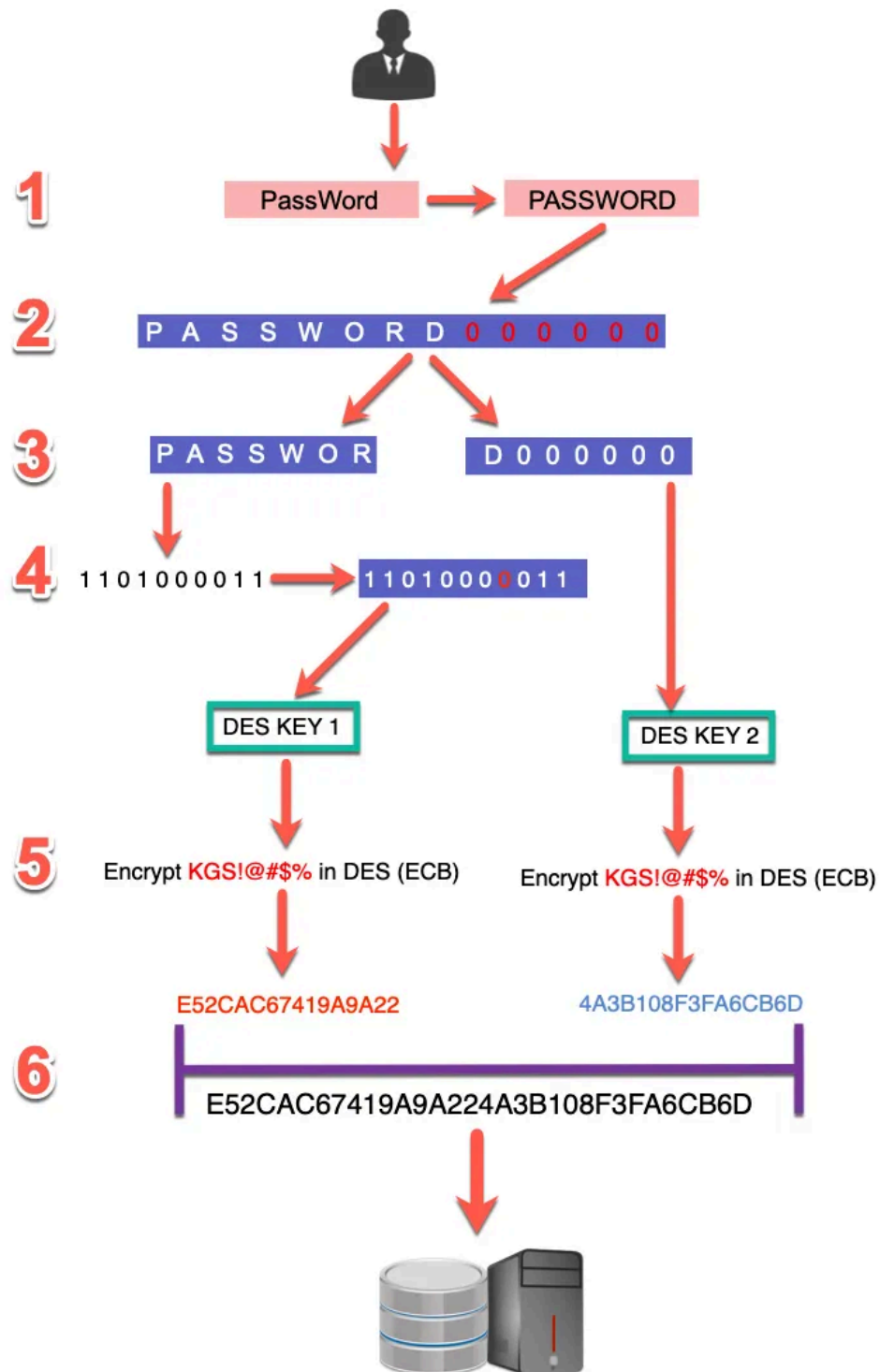
```
| python -c 'from passlib.hash import lmhash;print lmhash.hash("password")'
```

Press enter or click to view image in full size



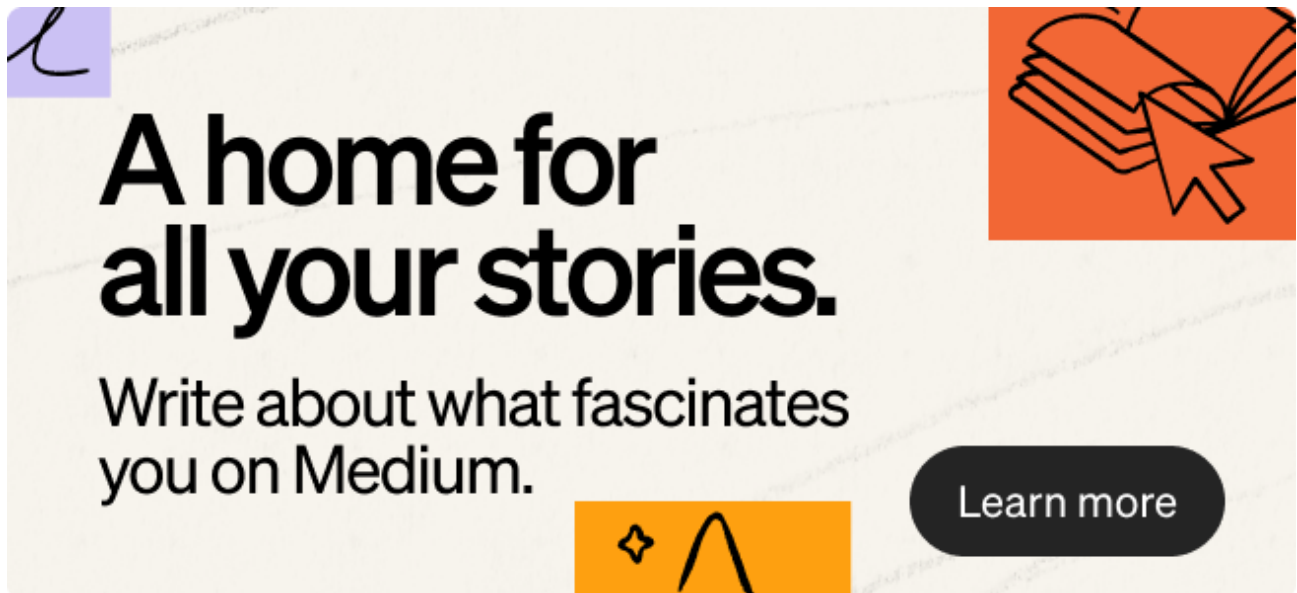
```
(jerbi@JERBI) - [~/ActiveD]  
$ python3 -c 'import passlib.hash; print (passlib.hash.lmhash.encrypt("PassWord"))'  
<string>:11: DeprecationWarning: the method passlib.handlers.windows.lmhash.encrypt() is deprecated as of Passlib 1.7, and will be removed in Passlib 2.0, use .hash() instead.  
e52cac67419a9a224a3b108f3fa6cb6d
```

Press enter or click to view image in full size



As you have already noticed, the first stage of the hash generation itself is a vulnerability, since :

Password1 , pAssword1 , PASSWORD1 , PassWord1 . . . ETC



All have the same hash !!!

Press enter or click to view image in full size

```
(jerbi@JERBI)-[~/ActiveD]
$ python3 -c 'import passlib.hash; print (passlib.hash.lmhash.encrypt("Password"))'
<string>:1: DeprecationWarning: the method passlib.handlers.windows.lmhash.encrypt() is deprecated as of Passlib 1.7, and will be removed in Passlib 2.0, use .hash() instead.
e52cac67419a9a224a3b108f3fa6cb6d

(jerbi@JERBI)-[~/ActiveD]
$ python3 -c 'import passlib.hash; print (passlib.hash.lmhash.encrypt("passWORD"))'
<string>:1: DeprecationWarning: the method passlib.handlers.windows.lmhash.encrypt() is deprecated as of Passlib 1.7, and will be removed in Passlib 2.0, use .hash() instead.
e52cac67419a9a224a3b108f3fa6cb6d
```

Let's assume a password like windowsseasy123The upper and lowercase combinations will be more than 32000 possibilities, and all of them will have the same hash!You can give it a try 😊

Another important thing to remember about LM is that **LM hash accepts only the 95 ASCII characters**, but yet **all lower case characters are converted to upper case**, which makes it only $95 - 26 = 69$ possibilities per character, which makes it just 7.5 trillion possibilities for each half instead of the total of 69^{14} for the whole minimum 14 characters..

Rainbow tables already exist containing all these possibilities, so cracking Lan Manager (LM) hashes isn't a problem at all.

Moreover, in case that the password is seven characters or less, the attacker doesn't need to brute force the 2nd half as it has the fixed value of **AAD3B435B51404EE**.

AAD3B435B51404EE : That's a representation for a Blank hash in LM.

Press enter or click to view image in full size

AAD3B435B51404EE

Je ne suis pas un robot

reCAPTCHA

Confidentialité - Conditions

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
AAD3B435B51404EE	LM	

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

NTHash (New Technology Lan Manager HASH)

A common misnomer is that NT hashes are really named NTLM hashes. They are not. **NTLM is network authentication protocol that uses NT hashes as its password hash algorithm.** The Kerberos protocol also uses NT hashes.

NTHash is the **currently used algorithm for storing passwords on windows systems.**

NTHashes of passwords are stored in **NTDS.dit** (database for Active Directory environments), also in the Security Accounts Manager (SAM) database. You're going to need administrator access over the domain controller

Even though the LM Hash is disabled, some hash dumps will give you the following hash format where the **first part is LM Hash** and **the second part is NT Hash** :

aad3b435b51404eeaad3b435b51404ee:8118cb8789b3a147c790db402b016a08

Starting with Windows Vista and Windows Server 2008, **by default, only the NT hash is stored.**

During a pentest, you can extract or dump these hashes.

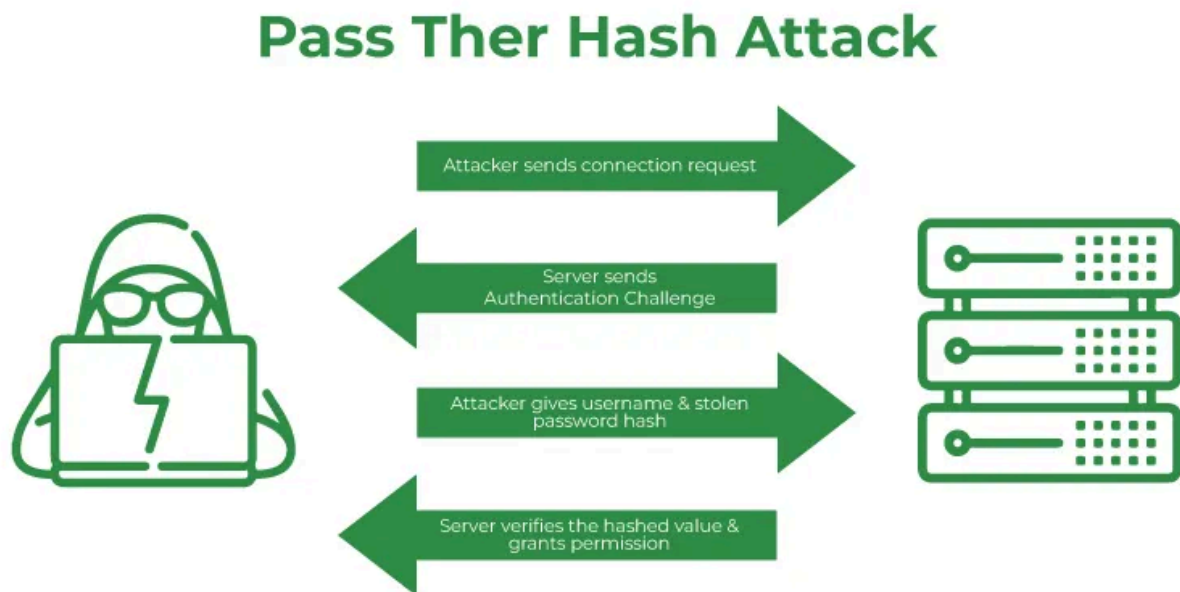
If it's a Windows machine, you can copy the **SAM Database** and dump the hashes from it using a tool like **impacket-secretsdump**.

If it's a Domain Controller and you have compromised it, you can dump these hashes by dumping the **ntds.dit** file inside, as that file includes passwords for all the users for all the machines in the Domain environment. Then after cracking, use the passwords to log in to other machines or do all the stuff you want.

If you are unable to crack the password, you can literally pass around the NT hashes or the LM:NT hashes using tools like **crackmapexec** or **impacket-psexec** or similar tools to

log in to other machines and do stuff. This is called **Pass-The-Hash** attack.

Press enter or click to view image in full size



NTHash generation

Converting a plaintext password into NTHash isn't complicated, it depends mainly on the **MD4 hashing algorithm**.

- 1 — The password is converted to Unicode
- 2 — MD4 is then used to convert it to the NTHash

Just like MD4(UTF-16-LE(password))

- 3 — Even in case of failing to crack the hash, it can be abused using Pass the hash technique as illustrated later.

Since there are no salts used while generating the hash, cracking NTHash can be done either by using pre-generated rainbow tables or using hashcat.

LET'S RECAP :.

-
1. **NT** and **LM Hashes** are used to **store password** in Windows machines or Domain Controllers.
 2. **NT** and **LM Hashes** can be found after dumping the **SAM Database** or **LSASS memory** of a Windows machine, dumping the **ntds.dit** file in a Domain Controller.
 3. **NT** and **LM Hashes** can be cracked to gain **cleartext password**. These can also be used for **pass-the-hash** attack, meaning trying these hashes with the usernames to log in to Windows machines.

4. The **NT hash** calculates the hash **based on the entire password** the user entered. The **LM hash splits the password into two 7-character chunks**, padding as necessary.

That's it for this article. In the next one, we will discuss **Net-NTLMv1/2 protocol used for network authentication**.

Reference :

https://www.linkedin.com/pulse/all-ways-your-nt-hash-can-stolen-roger-grimes-izml/?trk=article-ssr-frontend-pulse_more-articles_related-content-card